VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

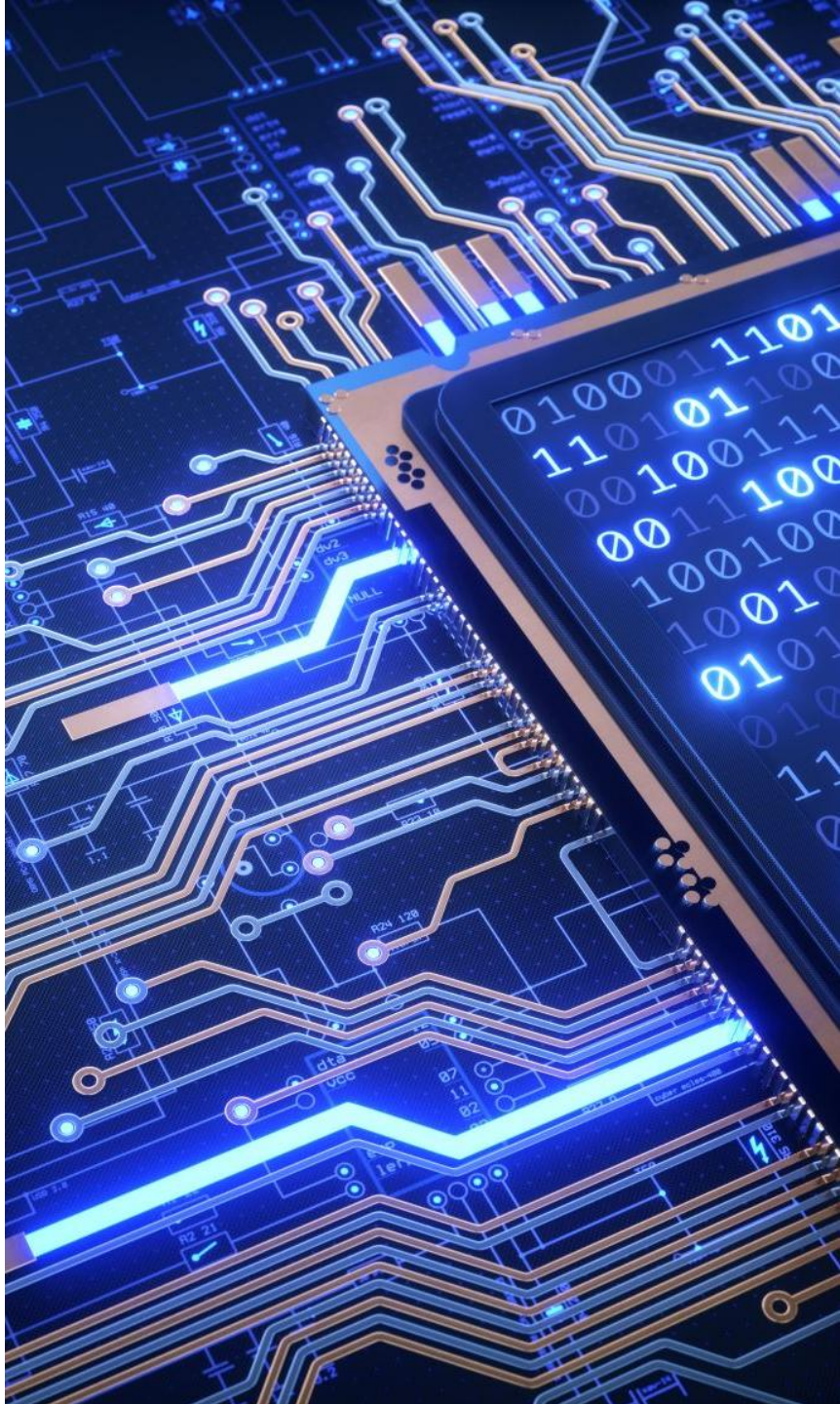
Digital Systems Design (BECE102L)

Dr Vishal Gupta

Assistant Professor

School of Electronics Engineering

Vellore Institute of Technology,
Vellore, Tamil Nadu, India



VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Module – 2

Verilog HDL

(Behavioral Modeling)

Behavioral Modeling in Verilog

- Specifies how a particular design should respond to a given set of input.
- Behavioral modelling uses structured procedures to specify the behaviour of the circuit.
- May be specified by Boolean expressions.
- Algorithms written in standard HLL like C

  Ex : {Carry,Sum} = a+b+c;

Key Concepts

- ✓ **Continuous Assignment:** Use assign statements for combinational logic.
- ✓ **Procedural Blocks:** Use always or initial blocks to describe behavior.
- ✓ **Sensitivity List:** Defines when the always block should be triggered.
- ✓ **Blocking vs. Non-blocking Assignments:** Use = for blocking and <= for non-blocking assignments.

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Behavioral Modeling in Verilog

- The **behavioral description is a powerful tool** to describe systems for which digital logic structures are not known or are hard to generate.
- Examples of such systems are complex arithmetic units, computer control units, and biological mechanisms that describe the physiological action of certain organs such as the kidney or heart.
- The behavioral description describes the system by showing how outputs behave with the changes in inputs. It focuses on the functionality and operation of the design rather than the implementation details.
- In this description, details of the logic diagram of the system are not needed; what is needed is how the output behaves in response to a change in the input.
- In Verilog, the major behavioral-description statements are **always** and **initial**.

Structured Procedures

- Two basic structured procedure statements or procedural blocks are
  - always (synthesizable)
  - initial (non-synthesizable)
- All behavioral statements can appear only inside these blocks.
- Each always or initial block has a separate activity flow (concurrency).
- There can be multiple always and initial blocks in a module.
- All procedural blocks start from simulation time 0.
- Can not be nested.

Structured Procedures: always statement

- Starts at time 0.
- Execute the statements in a looping fashion or **activity** that is **repeated continuously** in a digital circuit.
- Multiple always blocks, execute in parallel. All start at time 0.
- Equivalent to an **infinite loop.**
- **Stopped only** by $finish (close the environment) or $stop (interrupt).

Syntax:

```
always @(sensitivity list)
begin
// behavioral statements
end
```

Structure of the HDL Behavioral Description

Verilog Description

```
module half_add (I1, I2, O1, O2);
input I1, I2;
output O1, O2;
reg O1, O2;

/* Since O1 and O2 are outputs, they should be declared as
reg */

always @(I1, I2)

begin

#10 O1 = I1 ^ I2; // statement 1.
#10 O2 = I1 & I2; // statement 2.

/*The above two statements are signal-assignment
 statements with 10 simulation screen units Delay*/

/*Other behavioral (sequential) statements can be
added here*/
end
endmodule
```

- Referring to the Verilog code **always** is the Verilog behavioral statement.
- All Verilog statements inside **always** are treated as **concurrent**, the same as in the **data-flow description**.
- Also, here **any signal that is declared as an output** or appears at the left-hand side of a signal-assignment statement should be **declared as a register (reg) if it appears inside always**.
- O1 and O2 are declared **outputs, so they should also be declared as reg.**

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Behavioral Modelling – Half Adder

```
//Half Adder : Behavioral description
module ha_beh (x,y,s,c);
input x,y;
output s,c;
reg s, c;
always @(x or y)
begin
s = x ^ y;
c = x & y;
//{c,s} = x + y;
end
endmodule
```

Behavioral Modelling – Example

"always" block example:

```
module and_or_gate (out, in1, in2, in3);
  input     in1, in2, in3;
  output    out;
  reg       out;

  always @(in1 or in2 or in3) begin
     out = (in1 & in2) | in3;
  end

endmodule
```

"reg" type declaration. Not really a register in this case. Just a Verilog rule.

keyword

"sensitivity" list, triggers the action in the body.

brackets multiple statements (not necessary in this example.

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Behavioral Modelling – 2:1 Mux

```
//Behavioral modelling of 2:1 Mux
module mux (m_out, A, B, select);
output m_out;
input A, B, select;
reg m_out;
always @(A or B or select)
begin
  if (select = = 1)
    m_out = A;
  else
    m_out = B;
end
endmodule
```

Behavioral Modelling – T Flip Flop

```
//Behavioral modelling of T Flip-flop
module t_ff(clk,rst,t,q);
input clk,rst,t;
output reg q;
always@(posedge clk or posedge rst)
begin
if(rst == 1)
        q <= 0;
else
if (t == 0)
        q <= q;
else    q <= ~q;
end
endmodule
```

Test Bench: T Flip Flop

```
//Test Bench of T Flip-flop
module tb();
reg clk, rst, t;
wire q;
t_ff uut(clk, rst, t, q);
always
begin
#10 clk = ~clk;
end
initial
begin
clk = 0; rst = 0; t = 0;
#5 rst = 1;
#5 rst = 0; t = 0; #15 t = 1; #10 t = 0; #15 t = 1;
#5 rst = 1;
#5 rst = 0; t = 1;
#500 $stop;
end
endmodule
```

The initial Statement

- An initial block –

```
initial begin
  …
end
```

- It may consist of a **group of statements** (within begin … end) or **one statement.**
- Each initial block starts at **execution time 0.**
- All **initial blocks starts concurrently executing** and **finishes independent of each other**.
- Typically **used for** initialization, monitoring, waveforms and to execute one-time executable processes.

Sequential Statements

- There are several statements associated with behavioral descriptions.
- These statements have to **appear inside always or initial** in Verilog.
- The following sections discuss some of these statements.

1. IF Statement:

```
Verilog IF-Else Formats
if (Boolean Expression)
begin
   statement 1; /* if only one statement, begin and end
                   can be omitted */
   statement 2;
   statement 3;
.......
end
   else
begin
   statement a; /* if only one statement, begin and end
                   can be omitted */
   statement b;
   statement c;
.......
end
```

- **IF** is a **sequential statement** that appears inside **always** or **initial** in Verilog.
- The execution of IF statement is controlled by the Boolean expression.
- If the Boolean expression is true, then statements 1, 2, and 3 are executed.
- If the expression is false, statements a, b, and c are executed.

Contd…

Verilog

```
if (clk == 1'b1)
// 1'b1 means 1-bit binary number of value 1.
temp = s1;
else
temp = s2;
```

if clk is high (1), the value of s1 is assigned to the variable temp. Otherwise, s2 is assigned to the variable temp.

Contd…

EXECUTION OF IF AS ELSE-IF

Verilog

```
if (Boolean Expression1)
begin
   statement1; statement 2;.....
end
else if (Boolean expression2)
begin
   statementi; statementii;.....
end
else
begin
   statementa; statement b;....
   end
```

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

J-K Flip Flop using 'If' Statement

```
module jk_flip_flop (
input clk,
input reset,
input j,
input k,
output reg q);
always @(posedge clk or posedge reset)
begin
if (reset == 1)
begin
q <= 1'b0;
end
else
begin
if (j == 1'b0 && k == 1'b0)
 begin
 q <= q;
end
else
if (j == 1'b0 && k == 1'b1)
begin
q <= 1'b0;
end
else
if (j == 1'b1 && k == 1'b0)
begin
q <= 1'b1;
end
else
if (j == 1'b1 && k == 1'b1)
begin
 q <= ~q;
end
end
end
endmodule
```

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Contd…

2. The case Statement

The **case statement** is a sequential control statement.

Verilog Case Format

```
case (control-expression)
test value1 : begin statements1; end
test value2 : begin statements2; end
test value3 : begin statements3; end
default : begin default statements end
endcase
```

Verilog

```
case sel
2'b00 : temp = I1;
2'b01 : temp = I2;
2'b10 : temp = I3;
default : temp = I4;
endcase
```

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

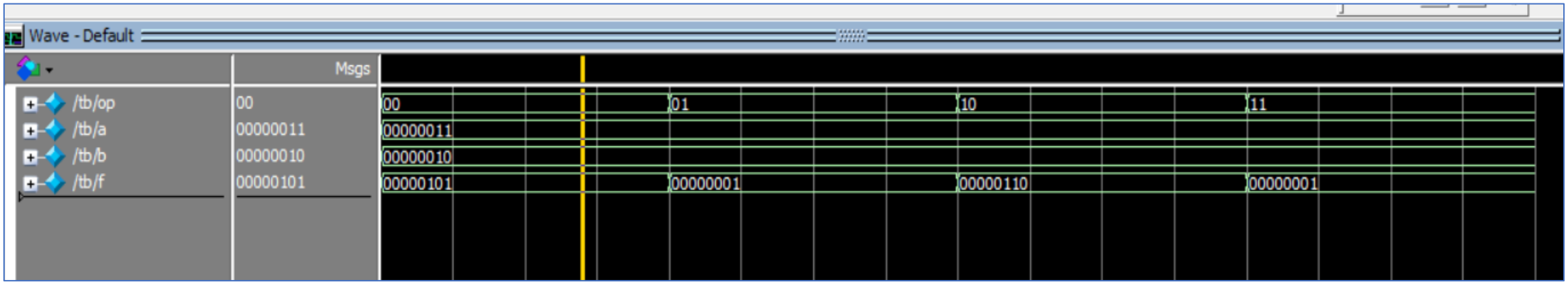
J-K Flip Flop using Case Statement

```
module jk_flip_flop (
input clk,
input reset,
input j,
input k,
output reg q);
always @(posedge clk or posedge reset)
begin
if (reset == 1)
begin
q <= 1'b0;
end
else
begin
case ({j, k})
  2'b00: begin  q <= q; end
  2'b01: begin  q <= 1'b0; end
  2'b10: begin  q <= 1'b1; end
  2'b11: begin q <= ~q; end
  default: begin  q <= 1'bx; end
endcase
end
end
endmodule
```

Example of case statement (ALU design)

```
module alu(f,a,b,op);
  input [1:0] op;  // operation [00: a+b; 01:a-b; 10: a*b; 11: a/b
  input [7:0] a,b;
  output reg [7:0] f;
  always@(*)
  begin
    case (op)
      2'b00: f<=a+b;
      2'b01: f<=a-b;
      2'b10: f<=a*b;
      2'b11: f<=a/b;
    endcase
  end
endmodule
```

```
module tb();
  reg [1:0] op;
  reg [7:0] a,b;
  wire [7:0] f;
  alu uut(f,a,b,op);
  initial
  begin
    a=8'b11; b=8'b10; op=2'b00;
    #20 op=2'b01;
    #20 op=2'b10;
    #20 op=2'b11;
    #20;
    $finish;
  end
endmodule
```



Contd…

The casex and casez Statements

casez:

- **It** does **not compare z-values** in the condition and case options.
- It ignores the z value.

casex :

- It does **not compare x** values in the condition and case options.
- It ignores the x value.

Example of casex statement

- **Priority encoder circuit**

```
module
casexexample(x,y,v,d);
  input [3:0]d;
  output reg x,y,v;
  always@(d)
  begin
    casex(d)
      4'b0000: {x,y,v}=3'bXX0;
      4'b0001: {x,y,v}=3'b001;
      4'b001X: {x,y,v}=3'b011;
      4'b01XX: {x,y,v}=3'b101;
      4'b1XXX: {x,y,v}=3'b111;
    endcase
  end
endmodule
```

```
//test bench
module tb();
  reg [3:0]d;
  wire x,y,v;
  casexexample uut(x,y,v,d);
  initial
  begin
    d=4'b0000;
    #10 d=4'b0001;
    #10 d=4'b0010;
    #10 d=4'b0011;
    #10 d=4'b0100;
    #10 d=4'b0111;
    #10 d=4'b1000;
    #10 d=4'b1010;
    $finish;
  end
endmodule
```

Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

Contd…

3. Wait or Dealy Statement

 # 10; // wait for 10 time instances i.e. 10 second or 10 ps etc.

Inter-statement delays: happen between statements, with a delay on the left side of an assignment

#5 c = a + b;

Intra-statement delays: occur within a single statement, evaluated at the current time, and the result is assigned after a delay on the right side

c = #5 (a + b);

Contd…

4. Loop statement

- Loop is a sequential statement that has to appear inside always or initial in Verilog.
- Loop is used to repeat the execution of statements written inside its body.
- The number of repetitions is controlled by the range of an index parameter.
- The loop allows the code to be compressed; instead of writing a block of code as individual statements, it can be written as one general statement that, if repeated, reproduces all statements in the block.

Contd…

- **for loop:**

```
for (i = 0; i <= 2; i = i + 1)
begin
if (temp[i] == 1'b1)
  begin
  result = result + 2**i;
  end
end
statement1;
statement2; ....
```

- **while loop:**

```
while (condition)
Statement1;
Statement2;
.....
end
```

```
while (i < x)
begin
i = i + 1;
z = i * z;
end
```

- **repeat:**

✓ In Verilog, the sequential statement repeat causes the execution of statements between its begin and end to be repeated a fixed number of times;

✓ no condition is allowed in repeat.

```
repeat (32)
begin
#100 i = i + 1;
end
```

- **forever:**

✓ The statement forever in Verilog repeats the loop endlessly. One common use for forever is to generate clocks in code-oriented test benches.

✓ The following code describes a clock with a period of 20 screen time units:

```
initial
begin
Clk = 1'b0;
forever #20 clk = ~clk;
end
```

Procedural Assignment

Two types: Blocking Assignments and Non-Blocking Assignments

1. Blocking Assignments (‘=’):

   Execute sequentially, meaning the next statement in the block won't execute until the current one is fully completed.

Ex: Given a = 4, b = 2, c = 1, d = 4, e = 2

```
// Combinational logic example (always @(*))
always @(*) begin
a = b + c;      // b + c is calculated, then assigned to a; a = 2 + 1 = 3
d = a - e;      // The new value of a is used in this calculation; d = 3 – 2 = 1
end
```

Procedural Assignment

- Non-Blocking Assignments ('<='):

  Evaluate all their right-hand sides (RHS) first and update the left-hand sides (LHS) at the end of the time step, allowing for simultaneous updates.

Ex: Given a = 4, b 2, c = 1, d = 4, e = 2

```
// Combinational logic example (always @(*))
always @(*) begin
a <= b + c;      // b + c is calculated, then assigned to a; a = 2 + 1 = 3
d <= a - e;      // The new value of a is used in this calculation; d = 4 – 2 = 2
end
```

Blocking Assignments

- It is denoted by "=" operator.
- In a sequential block it blocks any statement beyond it.

```
reg [7:0] a, b;
always @(posedge clk)
begin
    a = b + 2;
    b = a * 3;
end
```

Blocking assignment:

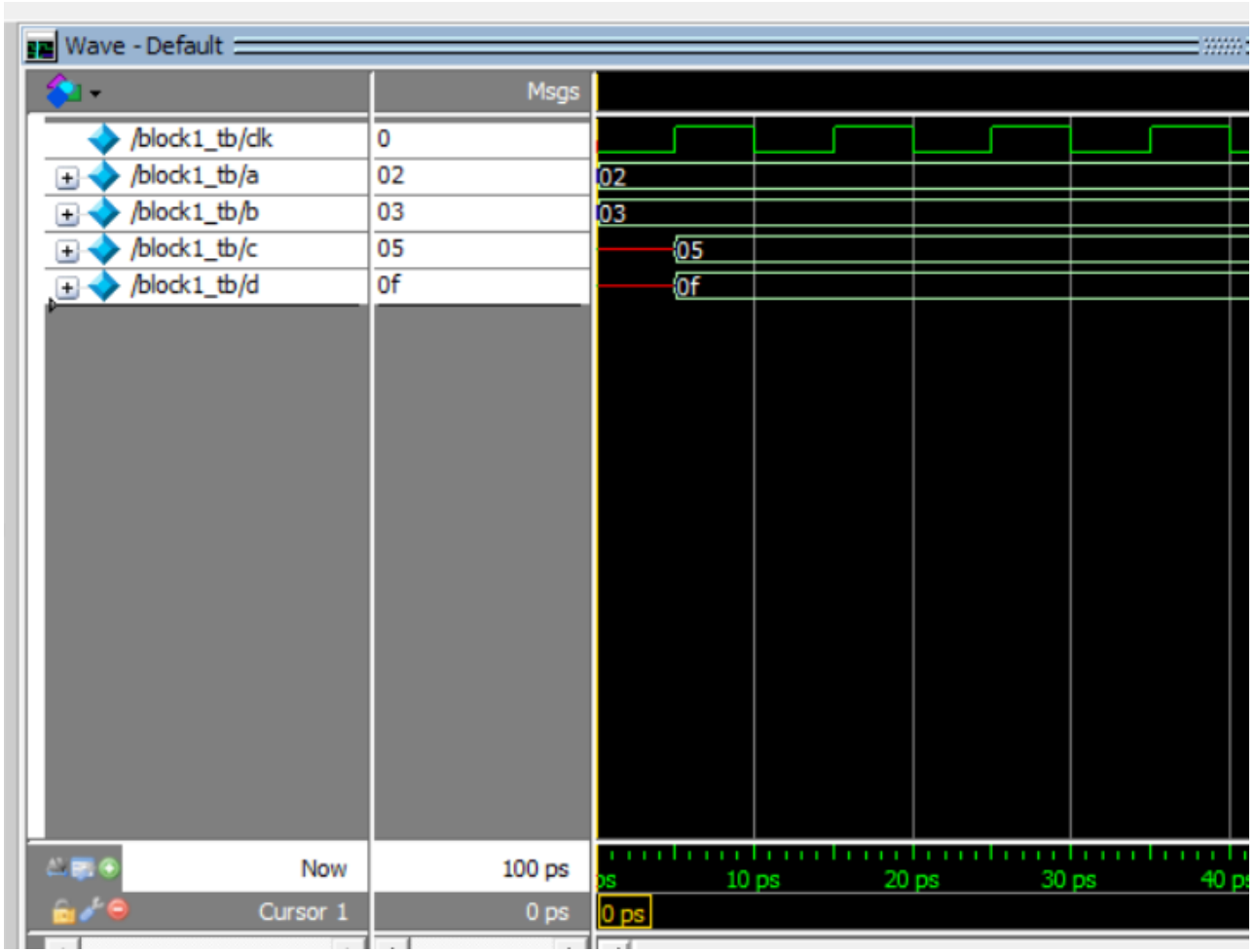
assignment on *a* MUST complete before assignment on *b* can start

Example 1

```
module
block1(c,d,a,b,clk);
  input clk;
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(posedge clk)
begin
  temp1 = a;
  temp2 = b;
  temp1 = temp2 + 2;
  temp2 = temp1 * 3;
  c = temp1;
  d = temp2;
end
endmodule
```

```
module block1_tb();
  reg clk;
  reg [7:0] a, b;
  wire [7:0] c,d;
  block1 uut(c,d,a,b,clk);
always
#5 clk=~clk;
initial
begin
  clk=0;  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```



Blocking Assignments

```
reg [7:0] a, b;
   always @(posedge clk)
   begin
   #10 a = b + 2;
   b = a * 3;
   end
```

1. **Wait 10;**
2. Read value of b and add 2 to it;
3. Assign the result to a;
4. Read the value of a and multiply by three;
5. Assign the result to b;

```
reg [7:0] a, b;
always @(posedge clk)
begin
      a = #10 b + 2;
   b = a * 3;
   end
```

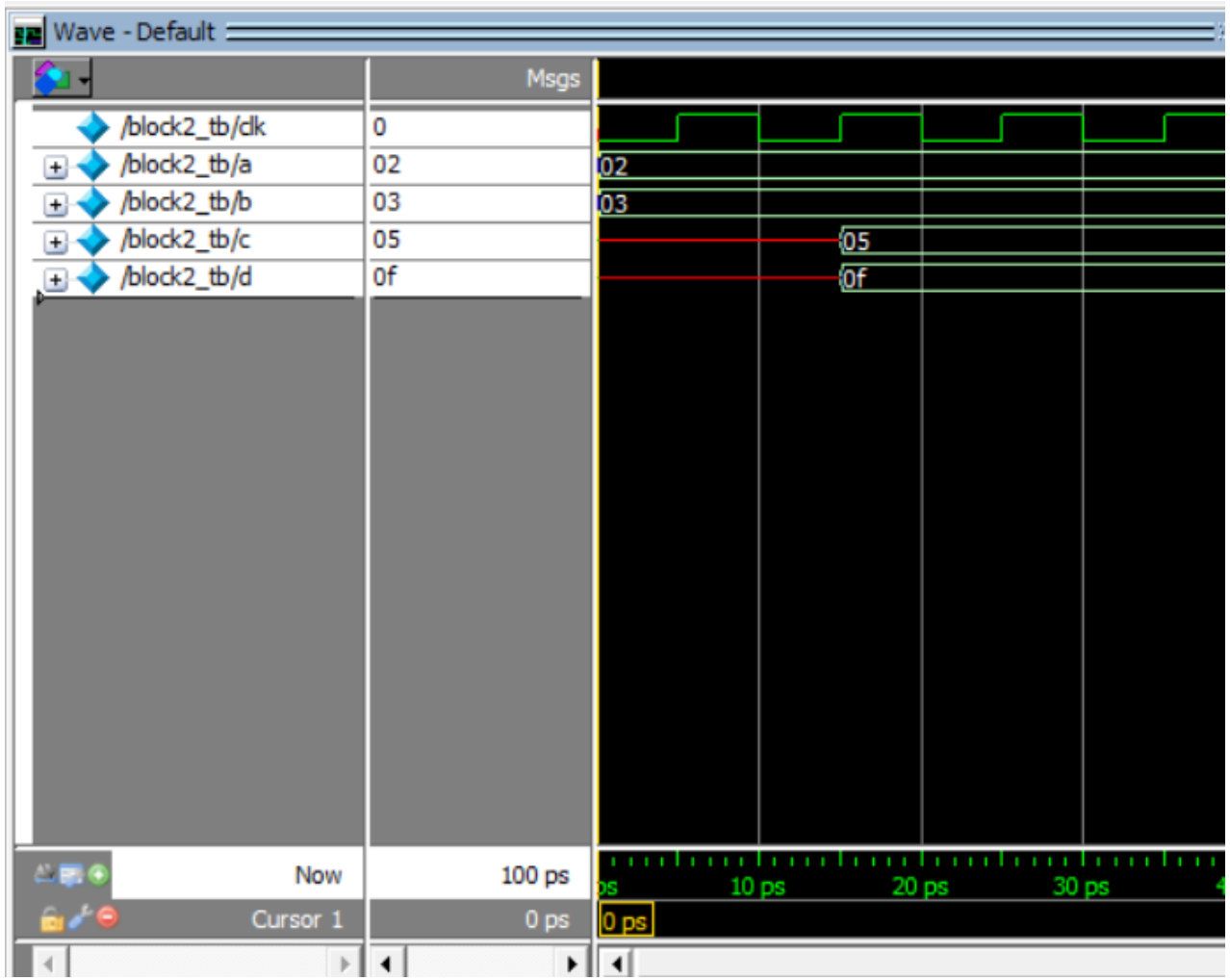
1. Read value of b and add 2 to it;
2. **Wait 10;**
3. Assign the result to a;
4. Read the value of a and multiply by three;
5. Assign the result to b;

Example 2

```
module
block2(c,d,a,b,clk);
  input clk;
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(posedge clk)
begin
  temp1=a;
  temp2=b;
  #10 temp1 = temp2 + 2;
  temp2 = temp1 * 3;
  c=temp1;
  d=temp2;
end
endmodule
```

```
module block2_tb();
  reg clk;
  reg [7:0] a, b;
  wire [7:0] c,d;
  block2 uut(c,d,a,b,clk);
always
#5 clk=~clk;
initial
begin
  clk=0;  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```

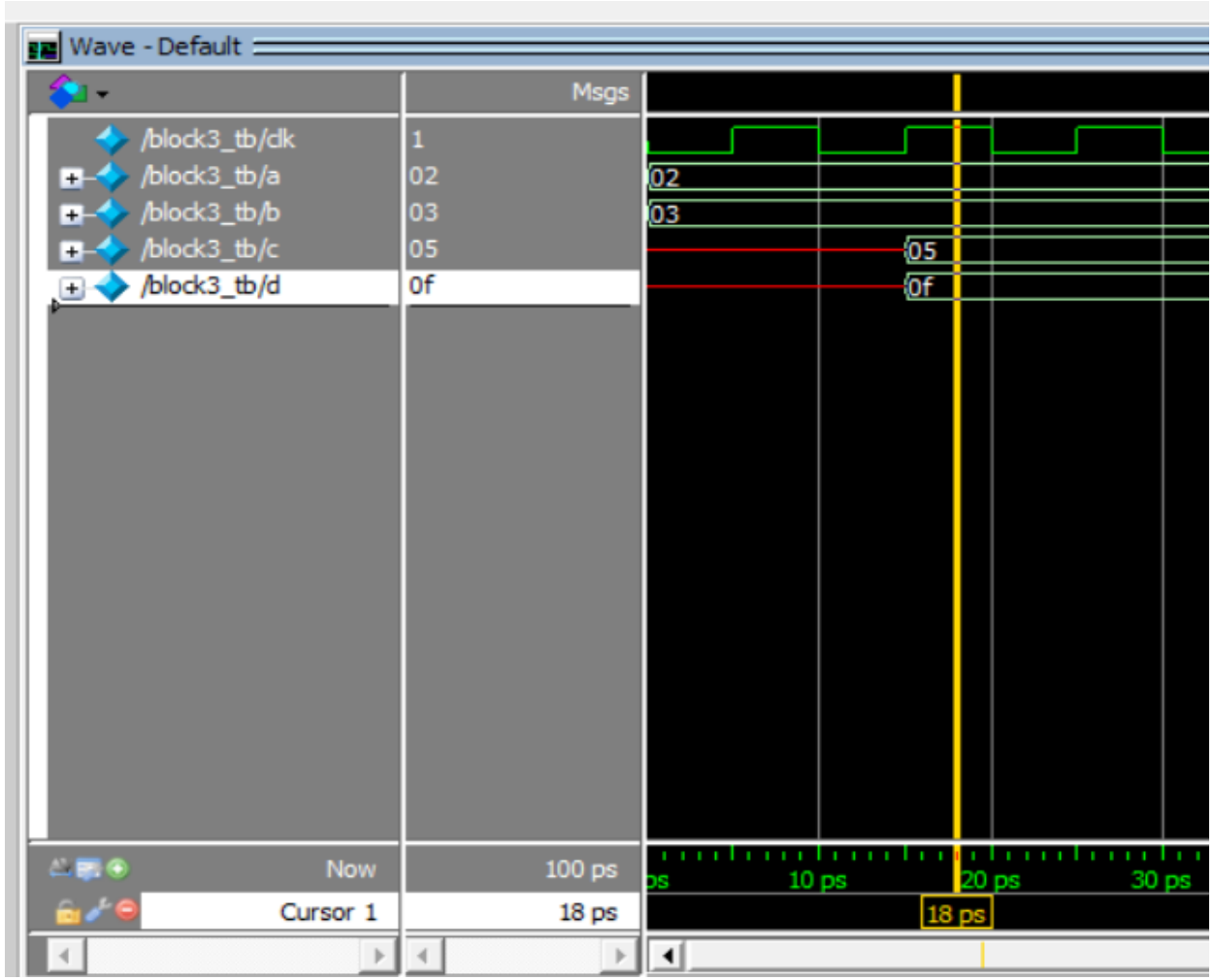


Example 3

```
module
block3(c,d,a,b,clk);
  input clk;
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(posedge clk)
begin
  temp1=a;
  temp2=b;
  temp1 = #10 temp2 + 2;
  temp2 = temp1 * 3;
  c=temp1;
  d=temp2;
end
endmodule
```

```
module block3_tb();
  reg clk;
  reg [7:0] a, b;
  wire [7:0]c,d;
  block3 uut(c,d,a,b,clk);
always
#5 clk=~clk;
initial
begin
  clk=0;  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```



Blocking Assignments

```
reg [7:0] a, b;
  always @(posedge clk)
  begin
  #5 a = #10 b + 2;
  #5 b = a * 3;
  end
```

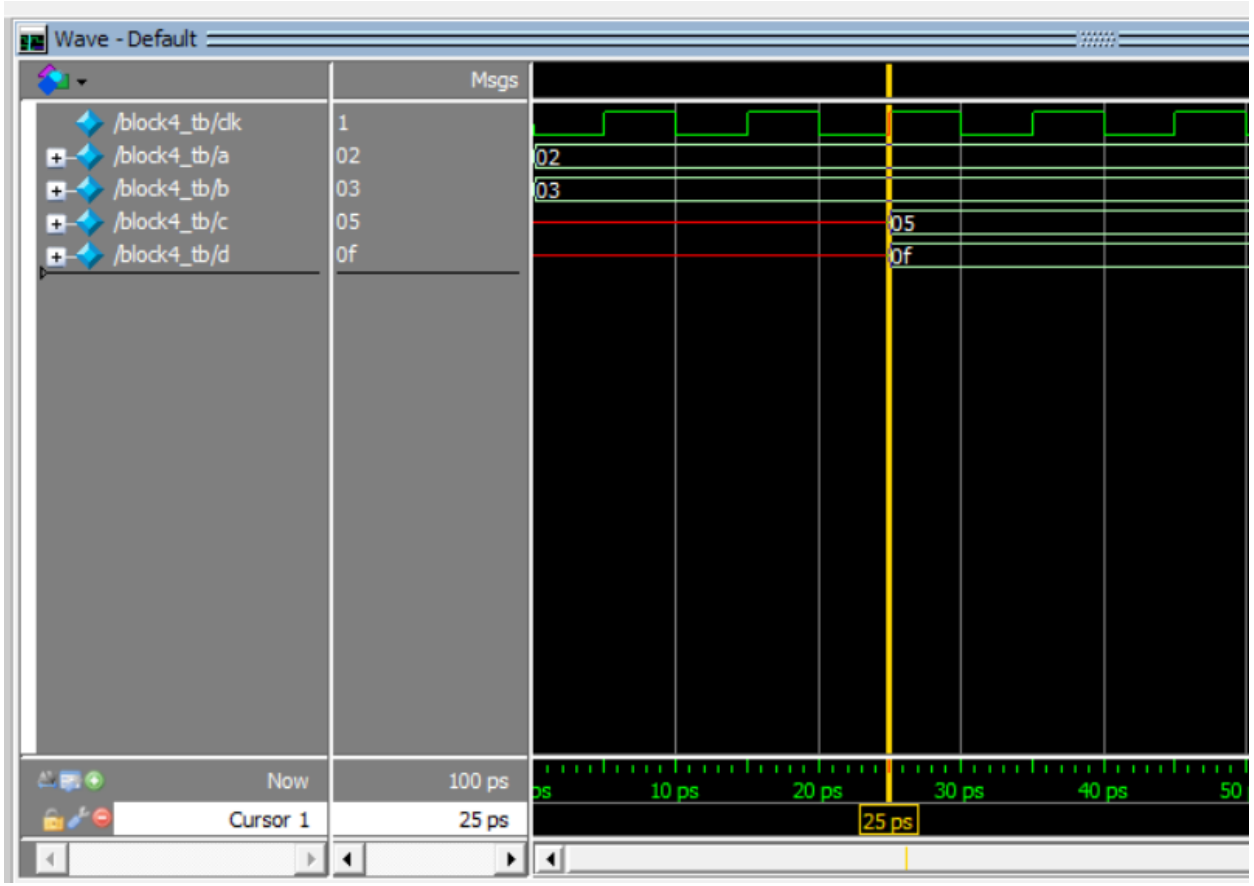
1. **Wait 5;**
2. Read value of b and add 2 to it;
3. **Wait 10;**
4. Assign the result to a;
5. **Wait 5;**
6. Read the value of a and multiply by three;
7. Assign the result to b;

Example 4

```
module block4(c,d,a,b,clk);
  input clk;
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(posedge clk)
begin
  temp1=a;
  temp2=b;
  #5 temp1 = #10 temp2 + 2;
  #5 temp2 = temp1 * 3;
  c=temp1;
  d=temp2;
end
endmodule
```

```
module block4_tb();
  reg clk;
  reg [7:0] a, b;
  wire [7:0]c,d;
  block4 uut(c,d,a,b,clk);
always
#5 clk=~clk;
initial
begin
  clk=0;  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```



Nonblocking Assignments

- These assignments **do not block execution of statements** that follow them in a sequential block.
- They are denoted by "<=" operator.
- The simulator processes all the events with a given timestamp according to the nature of the event.

❑ **Given the set of evaluate-events at time T**

❑ **First, do the following (in any order)**

- ✓ Evaluate the RHS of *all* assignments
- ✓ Assign the LHS of all procedural blocking assignments
- ✓ Assign the LHS of all continuous assignments
- ✓ Evaluate inputs and outputs of all primitives (gates)
- ✓ Print $display statements

❑ **Then, do the following (in any order)**

- ✓ Change the LHS of all nonblocking assignments

❑ **Then, do the following (in any order)**

- ✓ Print $monitor statements

Example: Effect of nonblocking statement

Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 4;

always @(a or b)
  begin
    a = b + 2;
    b = a * 3;
 end
```

a = 0 + 2 = 2
*b = 2 * 3 = 6*

Non-Blocking

```
reg [7:0] a, b;

initial b = 0;
initial a = 4;

always @(a or b)
  begin
    a <= b + 2;
    b <= a * 3;
 end
```

a = 0 + 2 = 2
*b = 4 * 3 = 12*

Non-Blocking Assignment means a and b are updated at the same time

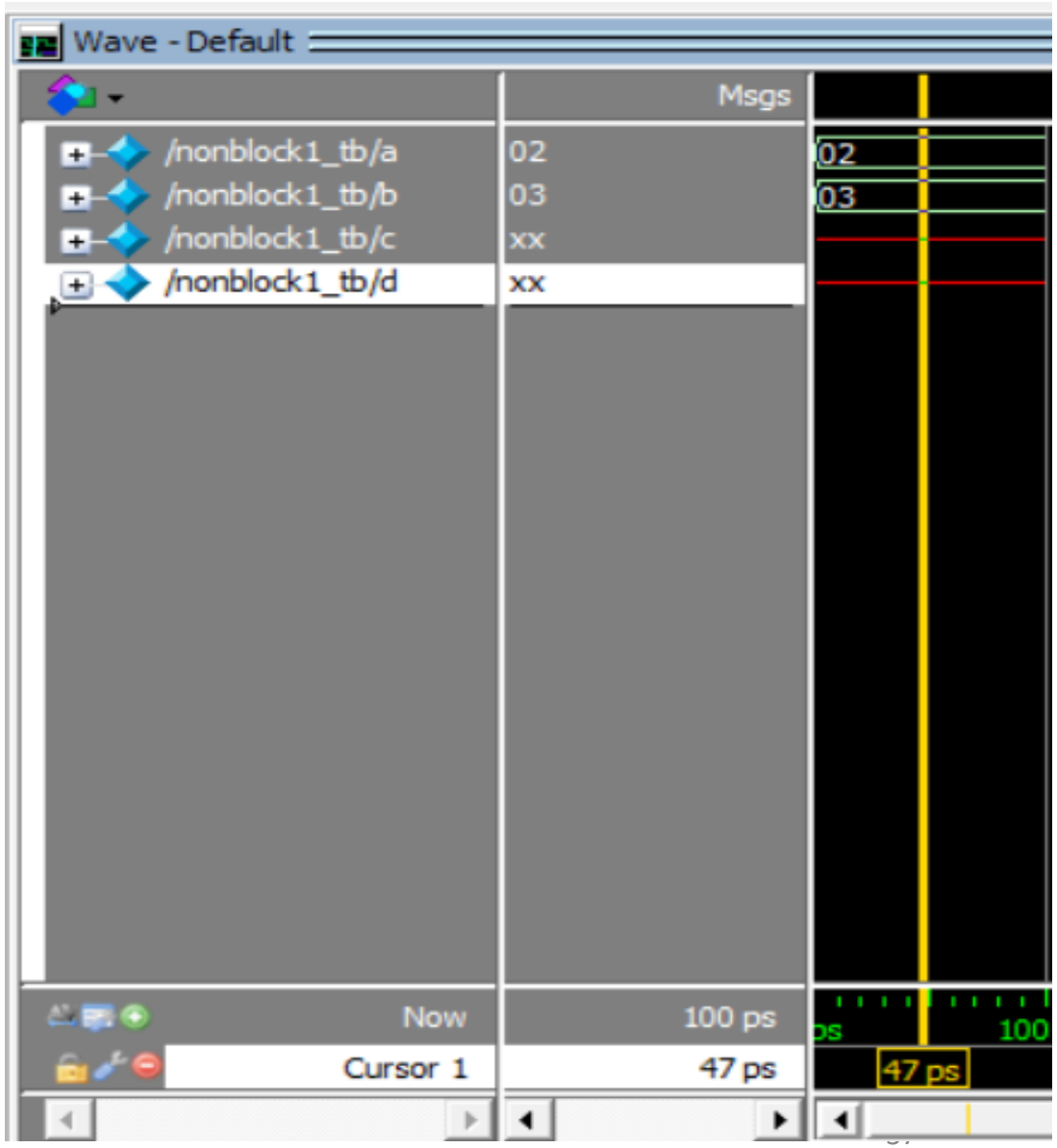
Example 4

```
module nonblock1(c,d,a,b);
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(a or b)
begin
  temp1<=a;
  temp2<=b;
  temp1 <= #10 temp2 + 2;
  temp2 <= #10 temp1 * 3;
  c<=temp1;
  d<=temp2;
end
endmodule
```

```
module nonblock1_tb();
  reg [7:0] a, b;
  wire [7:0] c,d;
  nonblock1 uut(c,d,a,b);

initial
begin
  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```



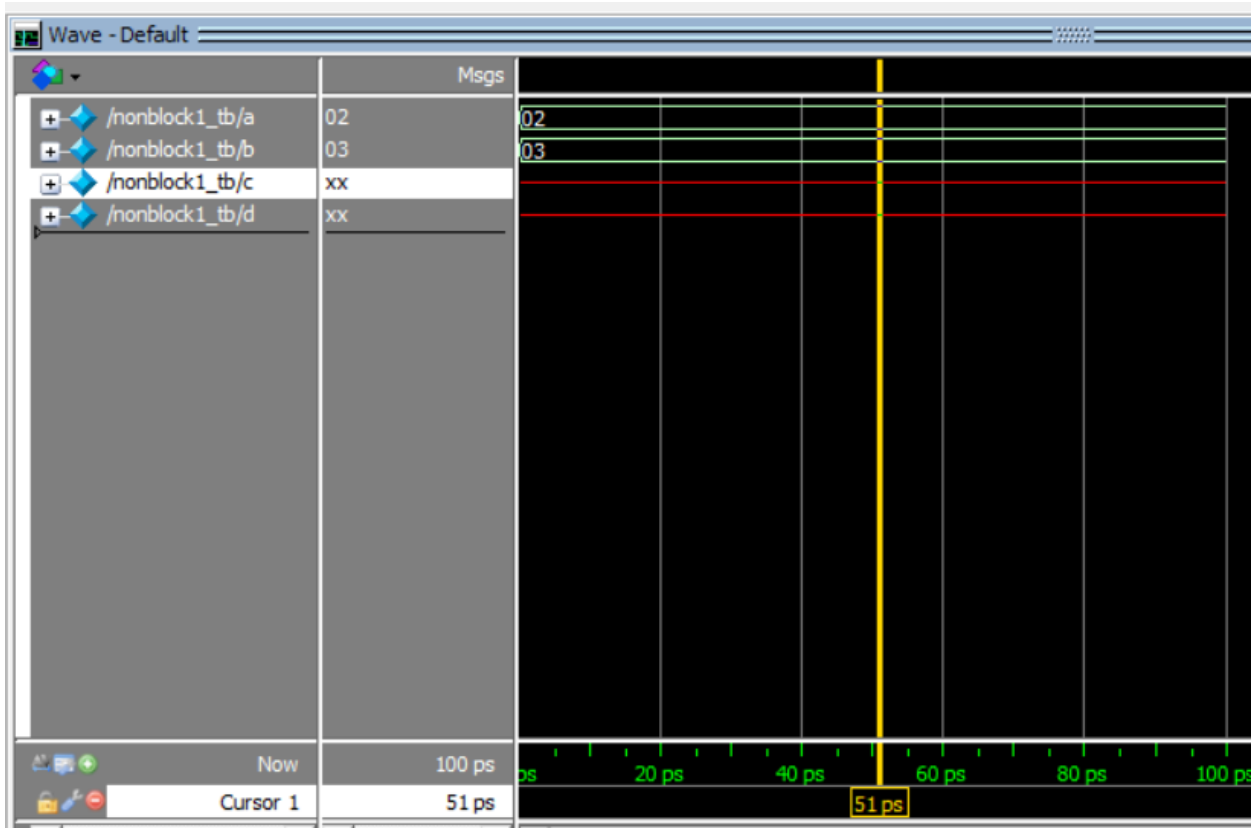
Example 5

```
module
nonblock1(c,d,a,b);
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(a or b)
begin
  temp1<=a;
  temp2<=b;
  temp1 <= #10 temp2 + 2;
  temp2 <= #10 temp1 * 3;
  c=temp1;
  d=temp2;
end
endmodule
```

```
module nonblock1_tb();
  reg [7:0] a, b;
  wire [7:0] c,d;
  nonblock1 uut(c,d,a,b);

initial
begin
  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```



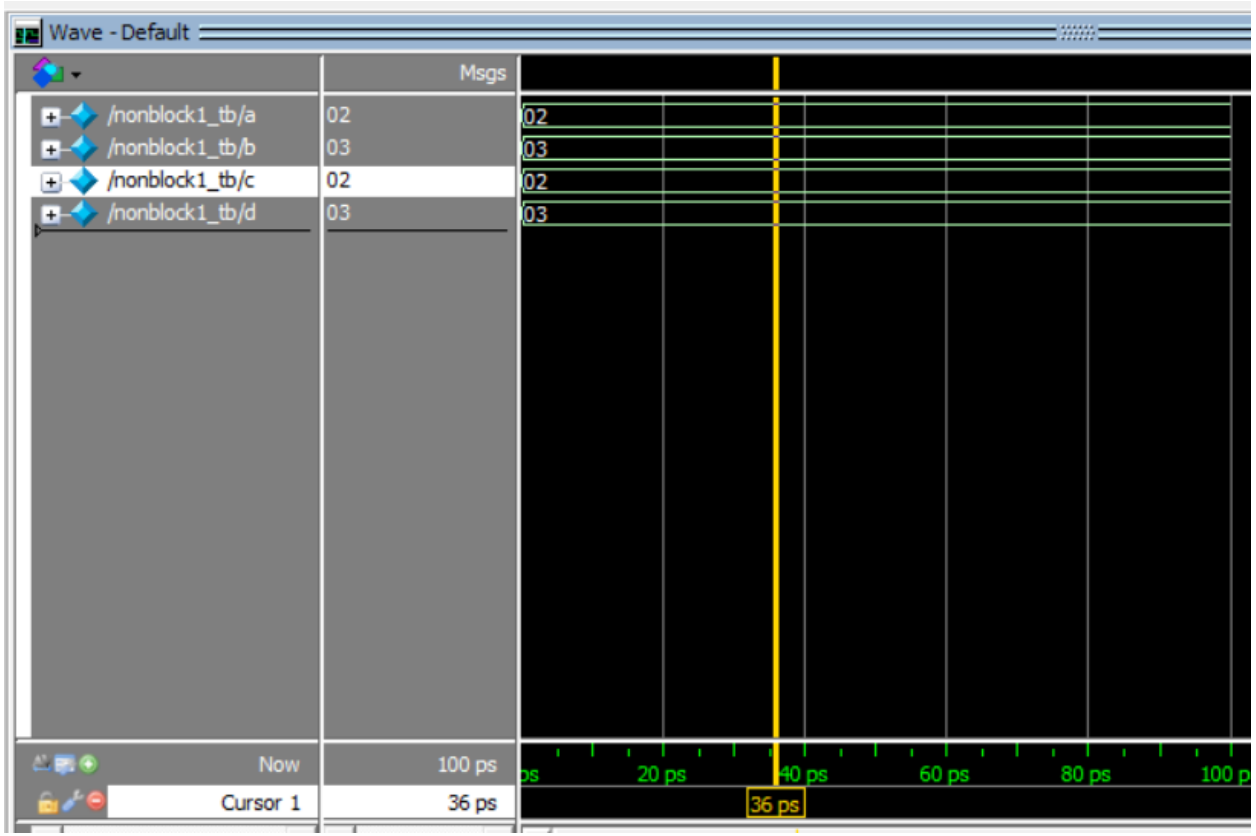
Example 6

```
module nonblock1(c,d,a,b);
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(a or b)
begin
  temp1=a;
  temp2=b;
  temp1 <= #10 temp2 + 2;
  temp2 <= #10 temp1 * 3;
  c=temp1;
  d=temp2;
end
endmodule
```

```
module nonblock1_tb();
  reg [7:0] a, b;
  wire [7:0] c,d;
  nonblock1 uut(c,d,a,b);

initial
begin
  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```



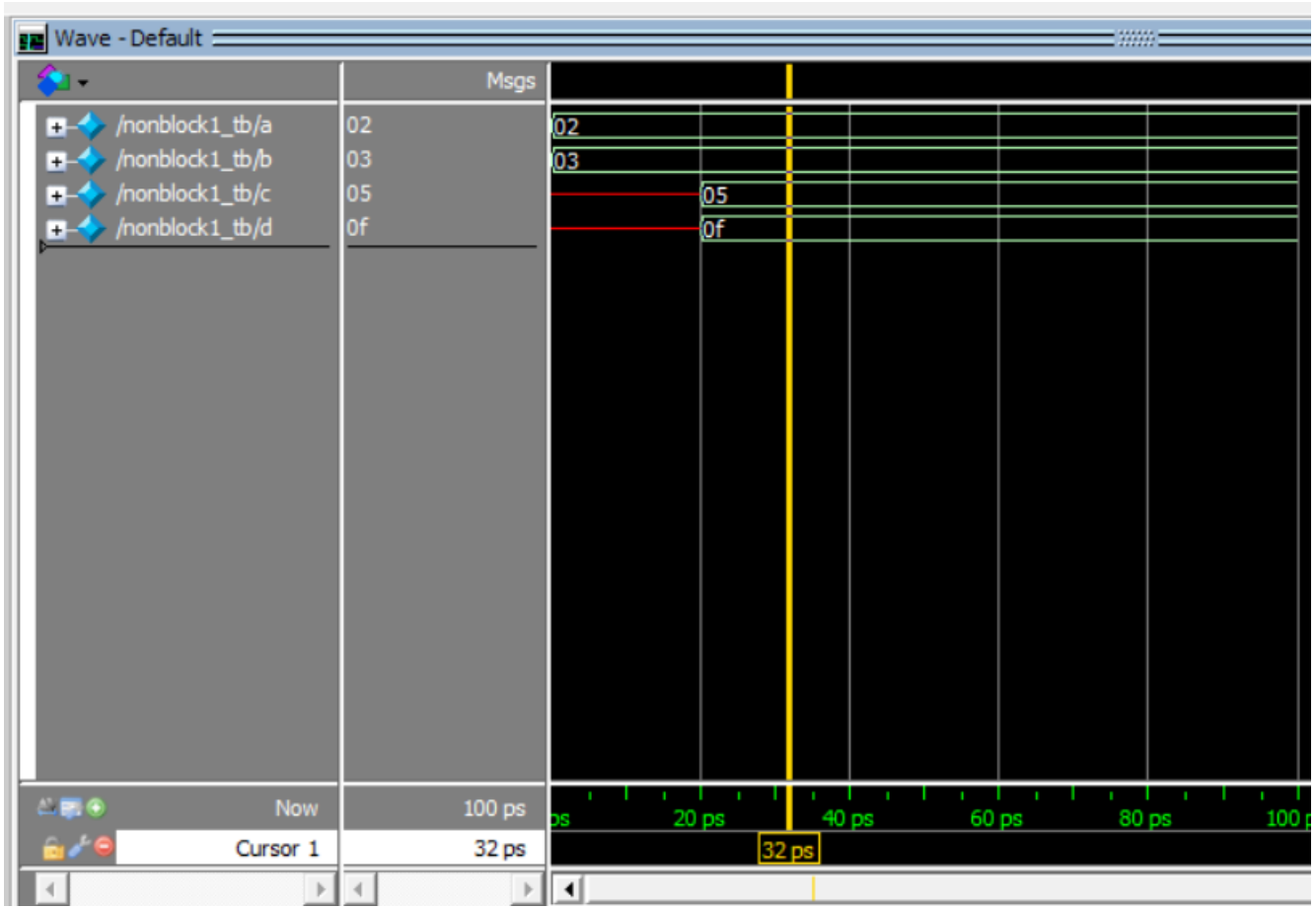
Example 7

```
module
nonblock1(c,d,a,b);
  input [7:0] a, b;
  output reg [7:0]c,d;
  reg [7:0] temp1,temp2;

always @(a or b)
begin
  temp1=a;
  temp2=b;
  temp1 = #10 temp2 + 2;
  temp2 = #10 temp1 * 3;
  c<=temp1;
  d<=temp2;
end
endmodule
```

```
module nonblock1_tb();
  reg [7:0] a, b;
  wire [7:0] c,d;
  nonblock1 uut(c,d,a,b);

initial
begin
  a=8'h2;   b=8'h3;
  #50;
end
endmodule
```

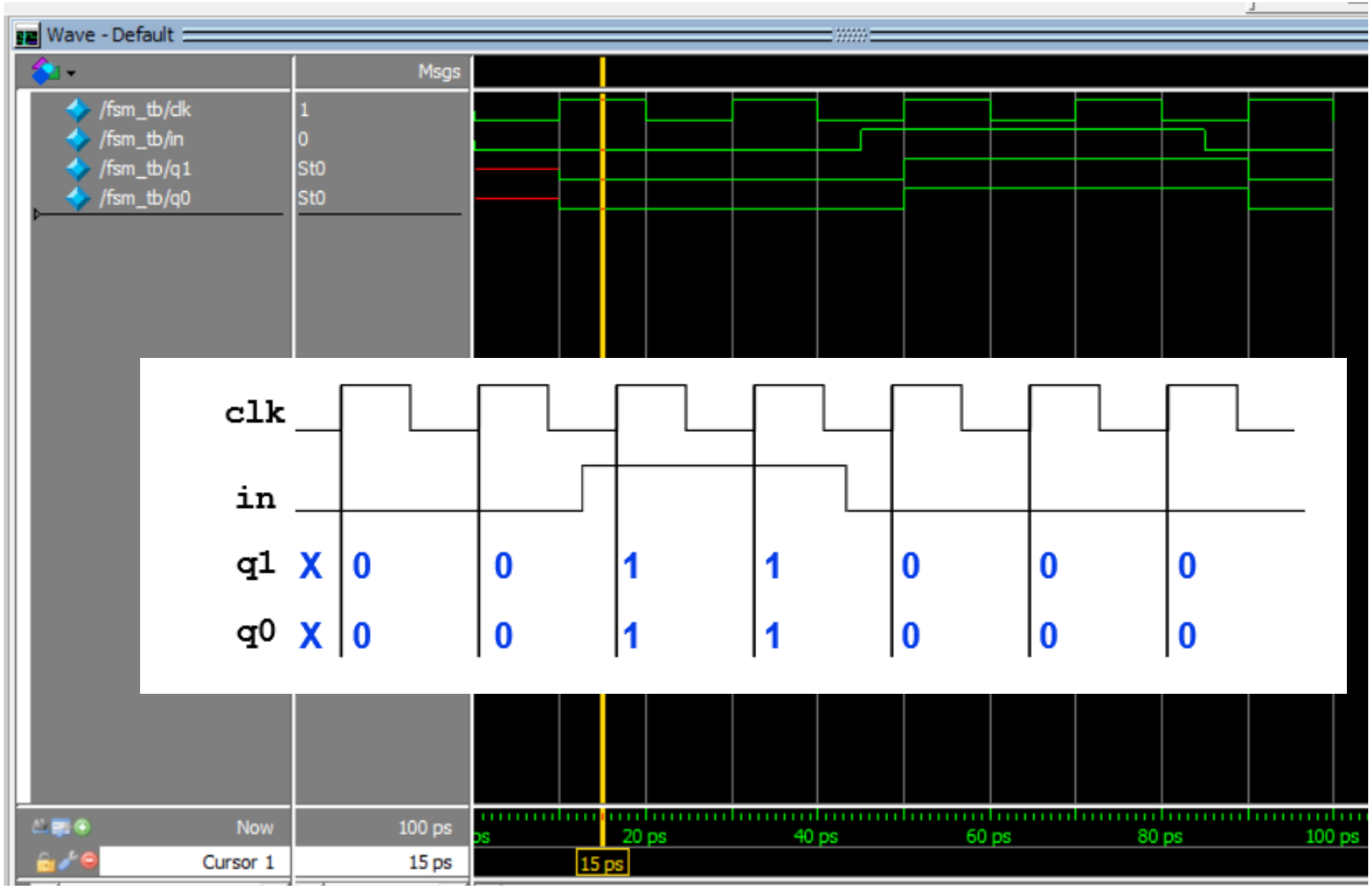


Example (blocking)

```
module fsm_mod(q1,q0,in,clk);
  input in,clk;
  output reg q1,q0;
  always @(posedge clk) begin
    q1=in;
    q0=in|q1;
  end
endmodule
```

```
module fsm_tb();
  reg in,clk;
  wire q1,q0;
  fsm_mod uut(q1,q0,in,clk);
    always
  #10 clk=~clk;

  initial begin
  clk=0; in=0;
  #45 in=1;
 #40 in=0;
 #100;
 $stop;
 end
 endmodule
```

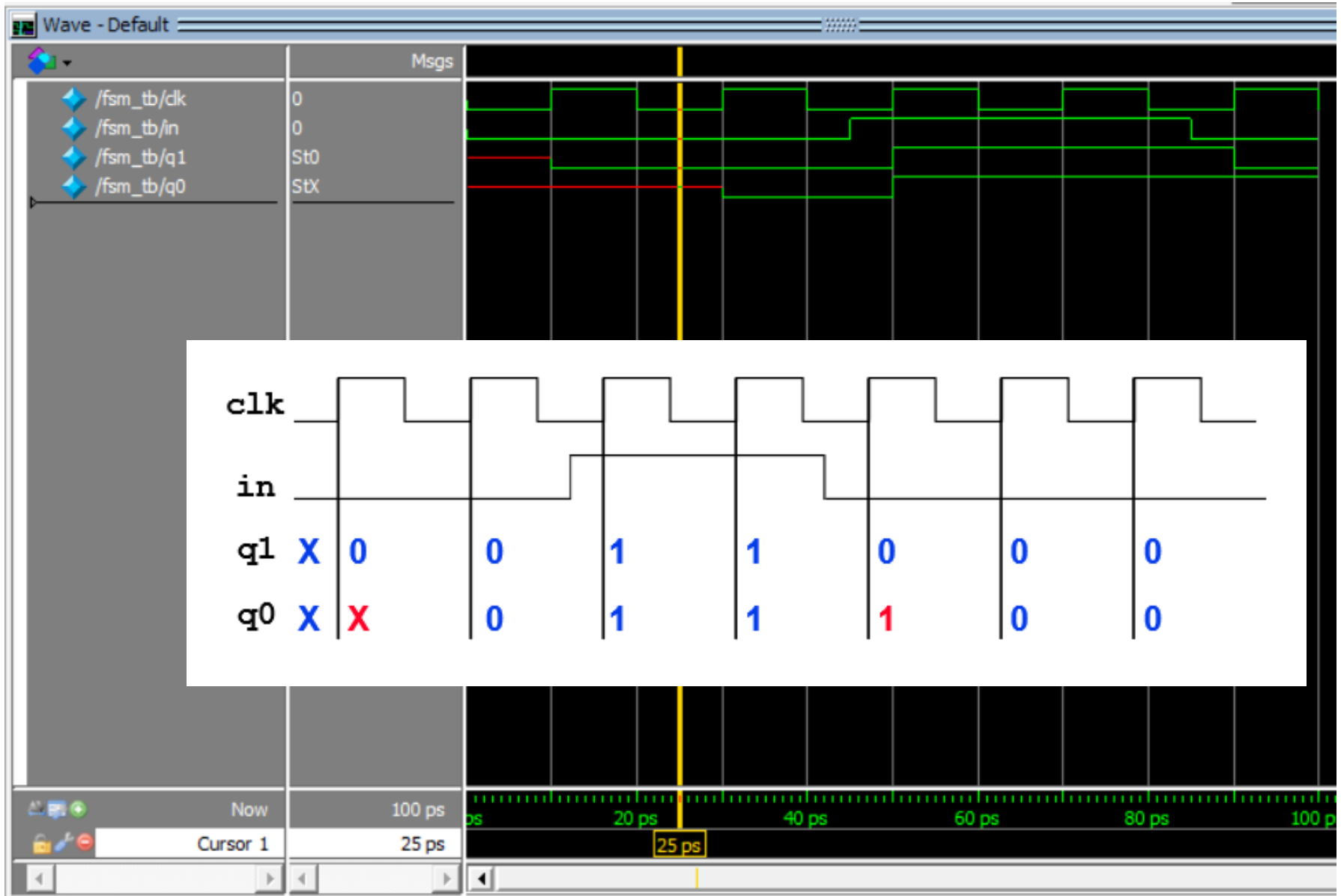


Example: (nonblocking)

```
module fsm_mod(q1,q0,in,clk);
  input in,clk;
  output reg q1,q0;
  always @(posedge clk) begin
    q1<=in;
    q0<=in|q1;
  end
endmodule
```

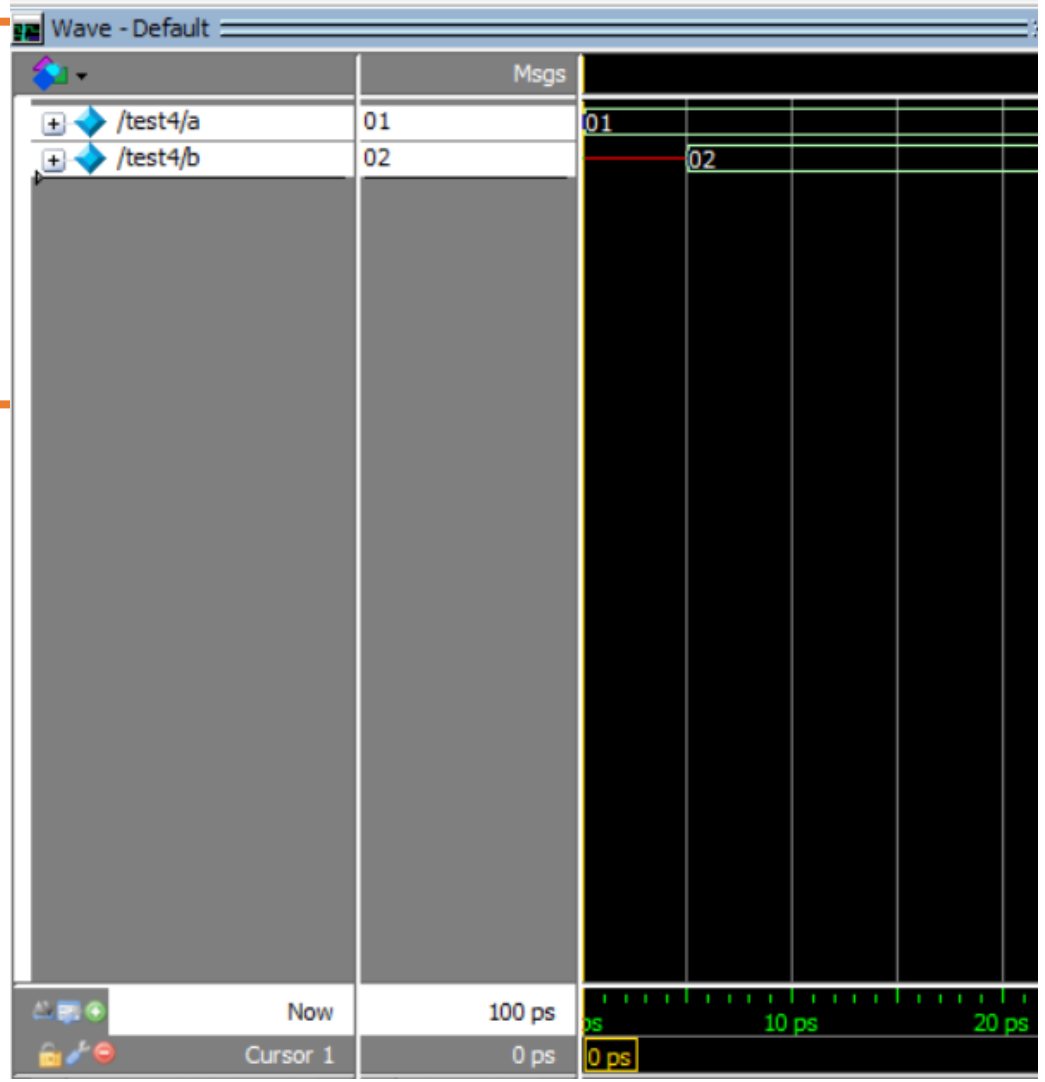
```
module fsm_tb();
  reg in,clk;
  wire q1,q0;
  fsm_mod uut(q1,q0,in,clk);
    always
  #10 clk=~clk;

  initial begin
  clk=0; in=0;
  #45 in=1;
 #40 in=0;
 #100;
 $stop;
 end
 endmodule
```



Delays and blocking assignments

```
initial
  begin
    a = 1;          t = 0: assign a with 1
    #5 b = a + 1;   t = 5: evaluate a + 1 and assign b
  end
```

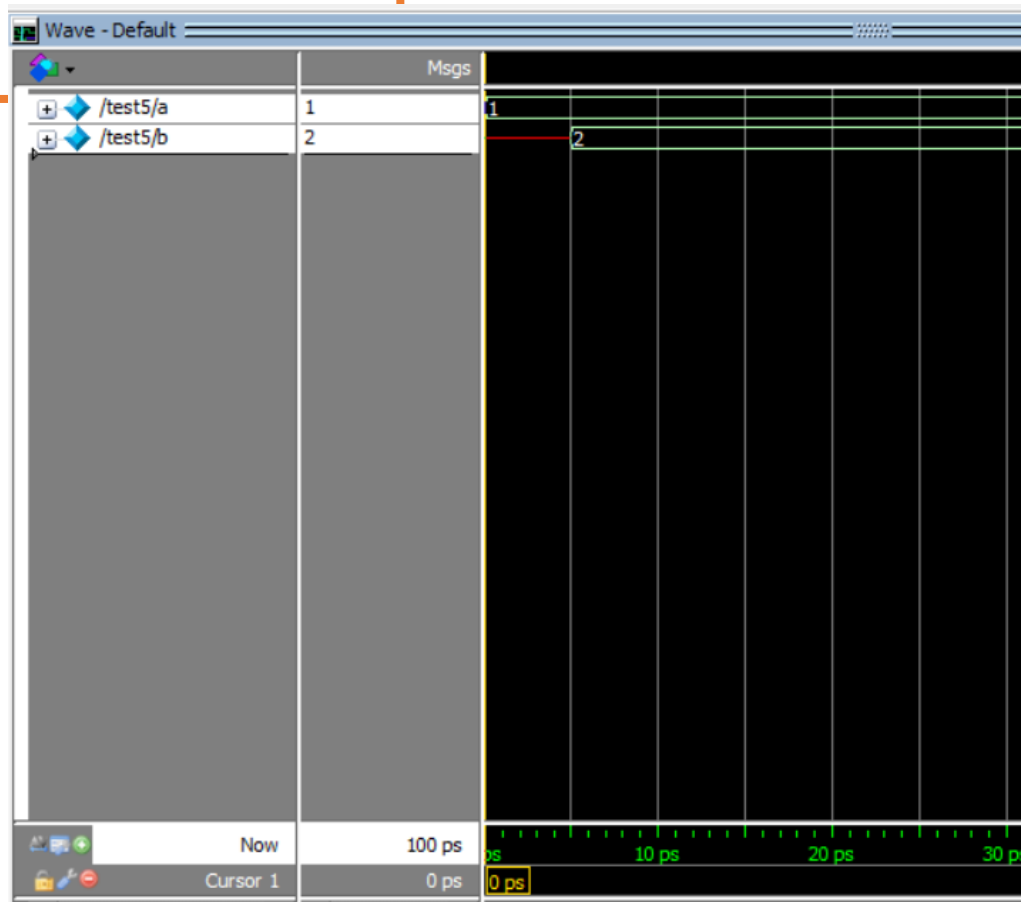



```
module test4();
 reg [3:0] a,b;
 initial
 begin
 a=1;
 #5 b=a+1;
  end
endmodule
```

Delays and nonblocking assignments

```
initial
  begin
    a <= 1;
    #5 b <= a + 1;
  end
```

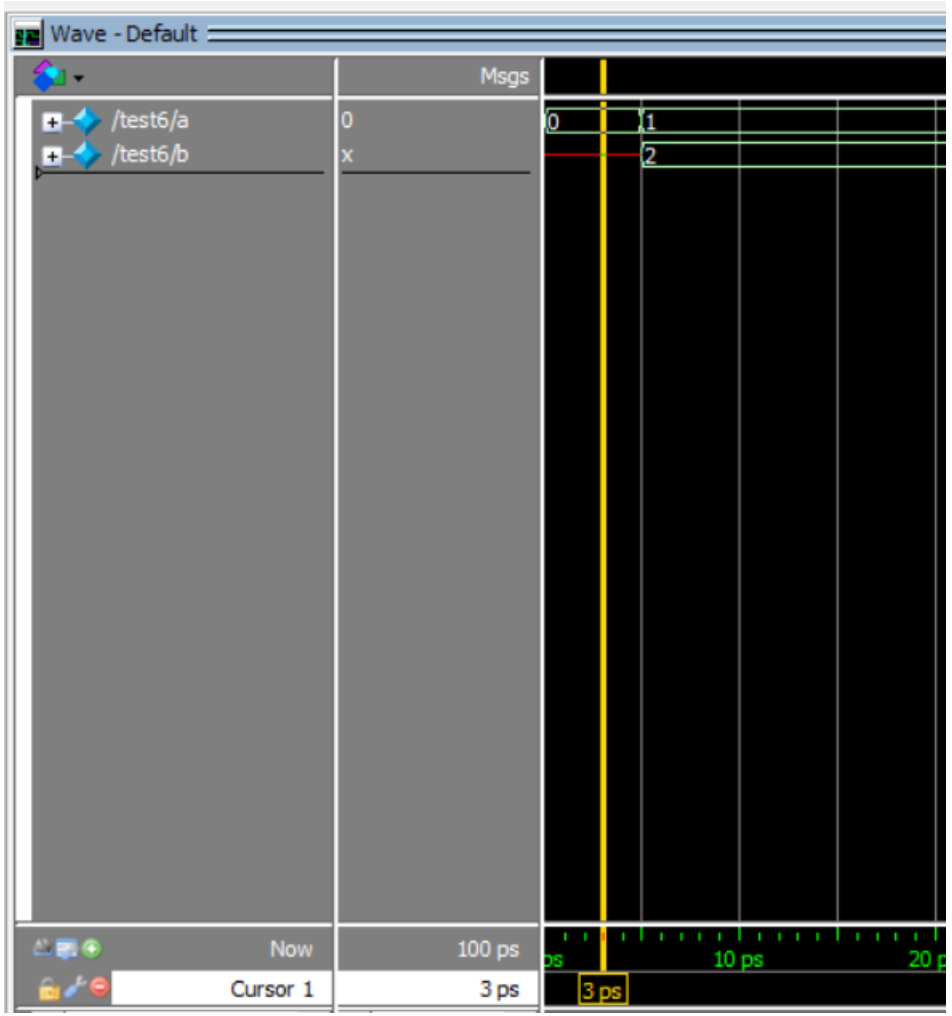
t = 0: assign a with 1 at the end of timestep 0
t = 5: evaluate a + 1 and assign b at the end of timestep 5




```
module test5();
 reg [3:0] a,b;
 initial
 begin
  a<= 1;
  #5 b<=a+1;
end
endmodule
```

Contd…

```
initial
  a = 0;
initial
  #5 a = a + 1;        t = 5: a is assigned with 1
initial
  #5 b = a + 1;        t = 5: b is indeterminate, because a changes
                              at t=5 using a blocking assignment
```




```
module test6();
 reg [3:0] a,b;
 initial
 a=0;
 initial
 #5 a=a+1;
 initial
 #5 b=a+1;
endmodule
```

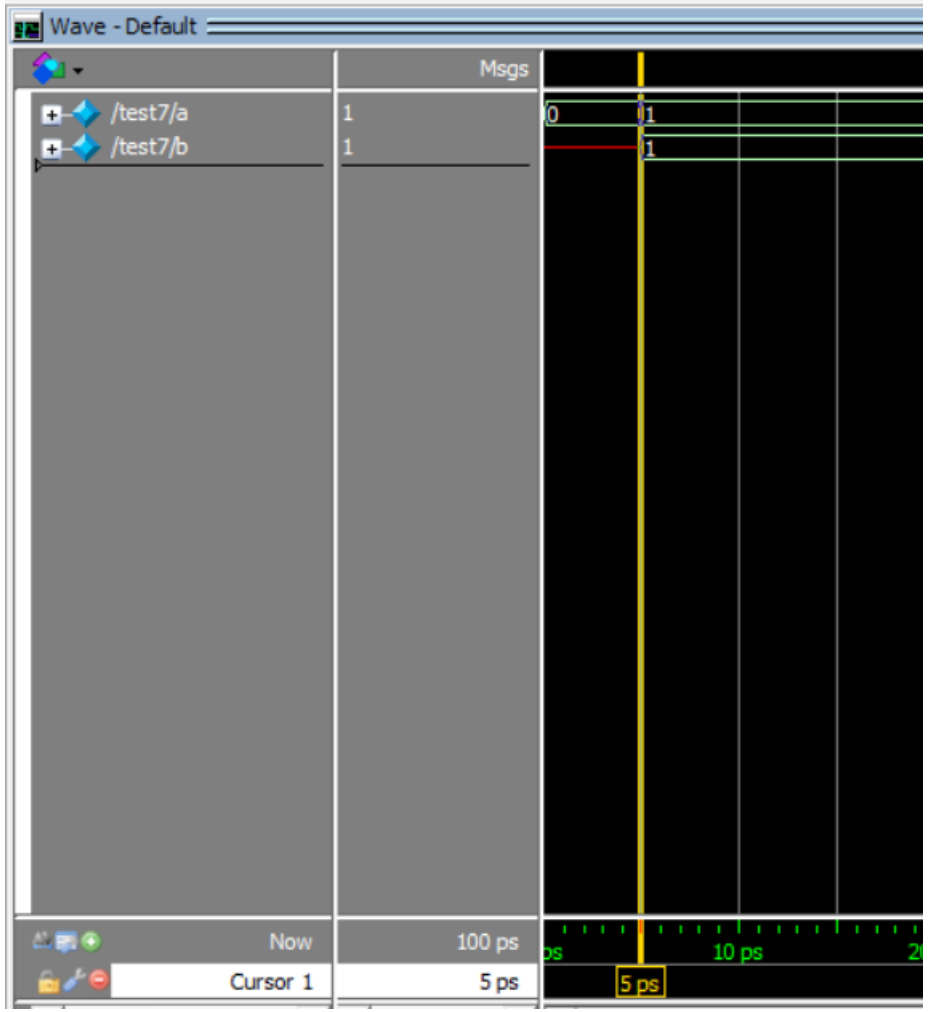
Contd…

```
initial
  a = 0;
initial
  #5 a <= a + 1;
initial
  #5 b <= a + 1;
```

t = 5: a is assigned with 1 at the end of t=5

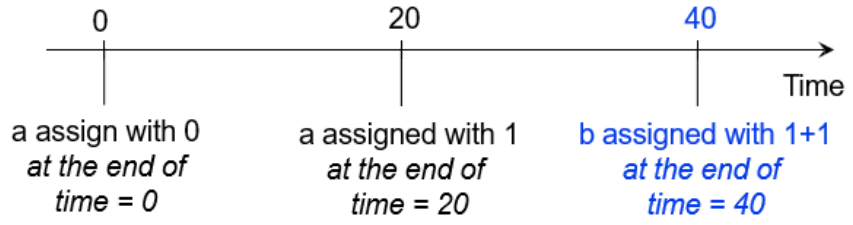
t = 5: b assigned with 1 at the end of t = 5

```
module test7();
 reg [3:0] a,b;
 initial
 a=0;
 initial
 #5 a <=a+1;
 initial
 #5 b<=a+1;
endmodule
```

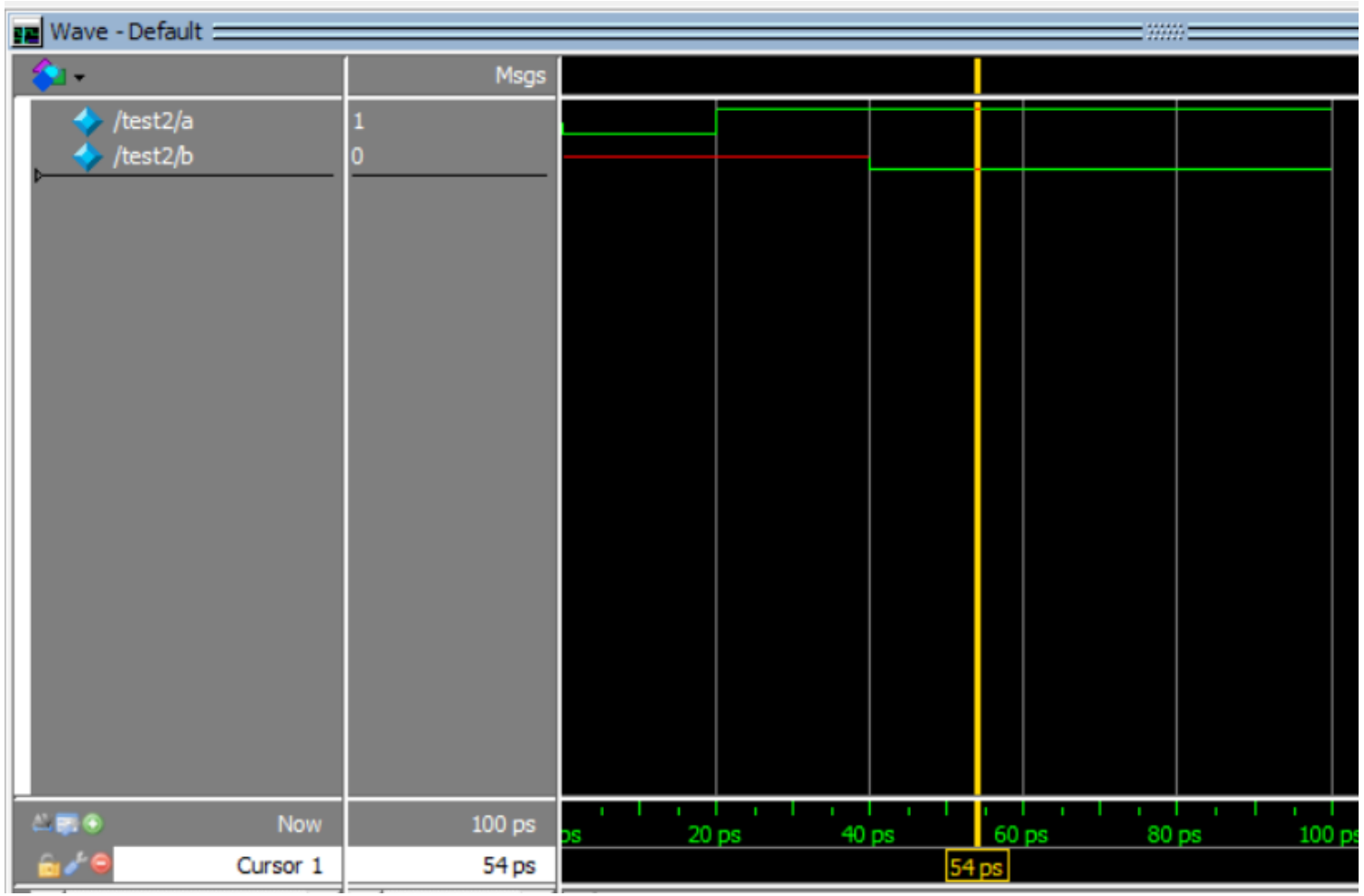


Contd…

```
initial
  begin
  a <= 0;
  #20 a <= 1;
  #20 b <= a + 1;
  end
```



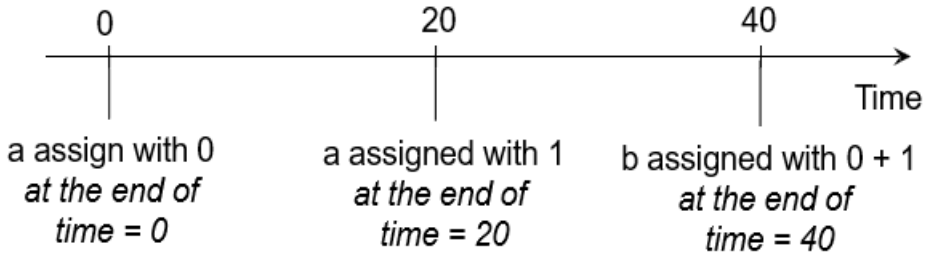

```
module test2();
 reg a,b;
 initial
 begin
a<=0;
 #20 a<=1;
 #20 b<=a+1;
 end
endmodule
```

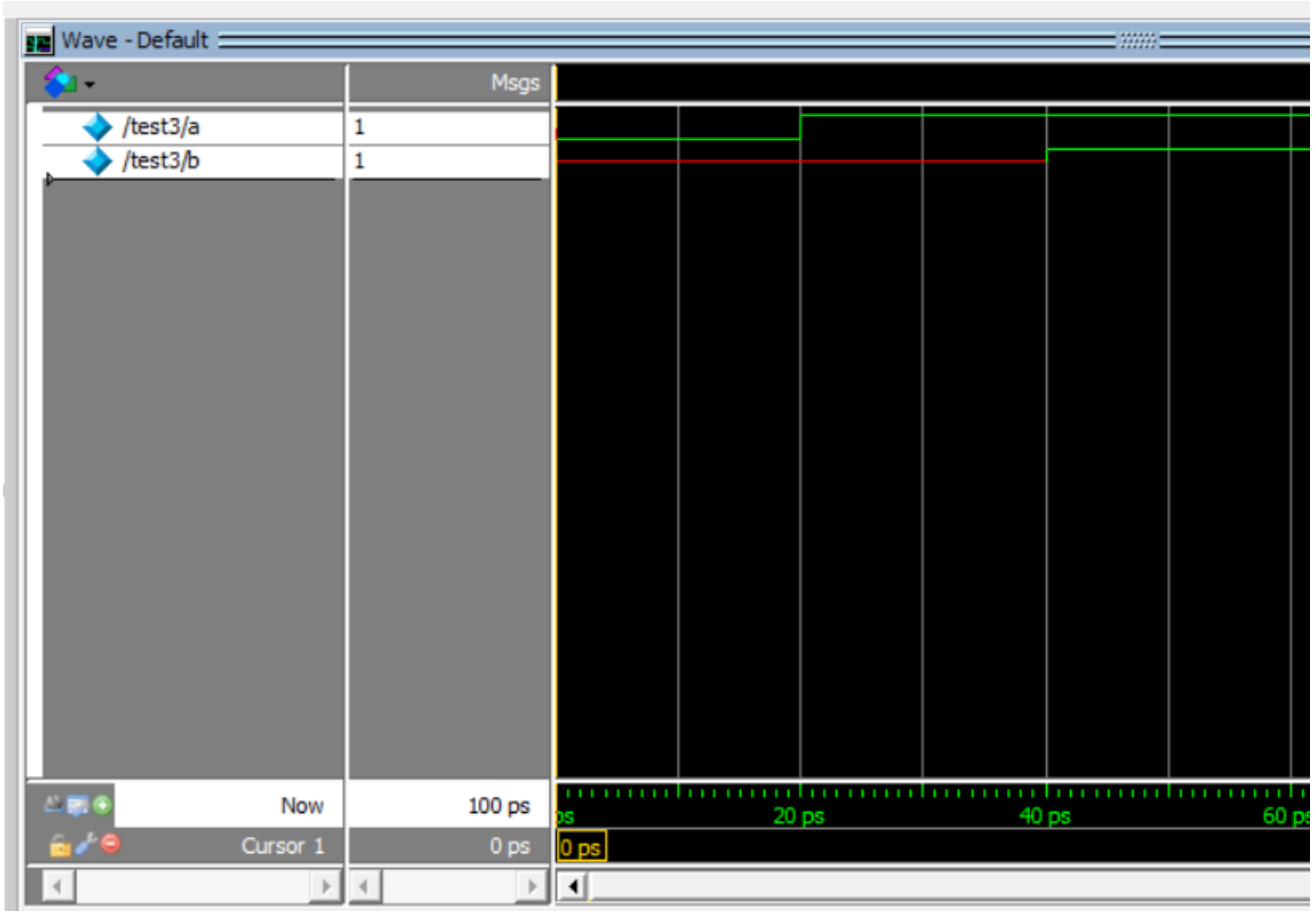


Contd…

```
initial
  begin
  a <= 0;
  #20 a <= 1;
      b <= #20 a + 1;
  end
```

1) This is one 'sequential block'
2) All these statements will execute at time t = 20.
3) b will only be assigned at *the end of* time = 40






```
module test3();
 reg a,b;
 initial
 begin
  a<=0;
   #20 a<=1;
  b<= #20 a+1;
 end
endmodule
```

Summary of blocking and non-blocking statements

- Commonly used to model registers in always blocks.
- Simulate the effect of multiple concurrent expressions.
- We will use non-blocking assignments to design hardware modules, FSM, data paths *etc* in a single always block.
- In practice **DO NOT MIX** non-blocking and blocking assignments in the same always block

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Any Queries!

VIT
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Thank you!